

Phase 2 Documentation

Goal

Run and compare NN training experiments with MLflow, then log and register the best ONNX model.

Steps

1. Load sensor datasets and validate shapes.
2. Build the NN and define the distance-constraint loss.
3. Train the model while logging metrics per epoch to MLflow.
4. Save the best weights, export ONNX, and log the model to MLflow.
5. Register the model in the MLflow Model Registry (Production stage).

MLflow Tracking Summary

- Experiments track NN training runs with different hyperparameters.
- Logged metrics include train/val loss and distance-error stats.
- Best epoch is selected from validation loss and exported as ONNX for deployment.
- The ONNX model is logged and registered in the MLflow Model Registry.

Deviation From The Spec

- No scikit-learn models and no regression task.
- The model is a PyTorch neural network trained with a distance-constraint loss.

```
In [1]: import os
import numpy as np
import polars as pl
import torch
import torch.nn as nn
import torch.optim as optim
import mlflow
import mlflow.pytorch
import mlflow.onnx
import onnx

In [2]: # =====
# CONFIG
# =====
SENSOR_DISTANCE = 0.5
BATCH_SIZE = 256
EPOCHS = 15
LEARNING_RATE = 1e-3
VAL_SPLIT = 0.2
SEED = 4

MLFLOW_TRACKING_URI = "http://localhost:5000/"
MLFLOW_EXPERIMENT = "akos-da"
MLFLOW_RUN_NAME = None

DATA_SAVE_PATH = "../data"
MODEL_SAVE_PATH = "../models"

SENSOR1_FILE = f"{DATA_SAVE_PATH}/m1_training.parquet"
SENSOR2_FILE = f"{DATA_SAVE_PATH}/m2_training.parquet"
SENSOR1_RESULTS_FILE = f"{DATA_SAVE_PATH}/m1_results.parquet"
SENSOR2_RESULTS_FILE = f"{DATA_SAVE_PATH}/m2_results.parquet"
AVG_DISTANCE_FILE = f"{DATA_SAVE_PATH}/avg_dist.parquet"

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
MOVE_DATA_TO_GPU = True
DATA_DEVICE = DEVICE if MOVE_DATA_TO_GPU else torch.device("cpu")
MAX_CPU_THREADS = os.cpu_count() or 1
torch.set_num_threads(MAX_CPU_THREADS)
torch.set_num_interop_threads(MAX_CPU_THREADS)
if torch.cuda.is_available():
    torch.backends.cudnn.benchmark = True
print(f"Using device: {DEVICE}, torch threads: {MAX_CPU_THREADS}")
print(f>Data device: {DATA_DEVICE}")

Using device: cuda, torch threads: 16
Data device: cuda

In [3]: # =====
# DATA LOADING
# =====
def load_data(file_path, device):
```



```

    if file_path.lower().endswith(".parquet"):
        df = pl.read_parquet(file_path)
    elif file_path.lower().endswith(".csv"):
        df = pl.read_csv(file_path, has_header=False)
    else:
        raise ValueError("Unsupported file format: " + file_path)

    if df.height == 0:
        raise ValueError(f"File {file_path} is empty")

    arr = np.array(df.to_numpy(), dtype=np.float32)
    return torch.tensor(arr, dtype=torch.float32, device=device)

sensor1_data = load_data(SENSOR1_FILE, DATA_DEVICE)
sensor2_data = load_data(SENSOR2_FILE, DATA_DEVICE)
sensor1_results = load_data(SENSOR1_RESULTS_FILE, DATA_DEVICE)
sensor2_results = load_data(SENSOR2_RESULTS_FILE, DATA_DEVICE)
avg_distance = load_data(AVG_DISTANCE_FILE, DATA_DEVICE)

if len(sensor1_data) != len(sensor2_data):
    raise ValueError("Both feature files must have the same number of rows")

if sensor1_data.shape[1] != sensor2_data.shape[1]:
    raise ValueError("Both feature files must have the same number of columns")

if len(sensor1_results) != len(sensor1_data):
    raise ValueError("Prediction target batch 1 must have the same number of rows as the first feature file")

if len(sensor2_results) != len(sensor2_data):
    raise ValueError("Prediction target batch 2 must have the same number of rows as the second feature file")

if len(avg_distance) != len(sensor1_data):
    raise ValueError("Distance labels must have the same number of rows as the feature files")

input_size = sensor1_data.shape[1]
print(f"input size {input_size}")

print(f"Loaded data: {len(sensor1_data)} samples, {input_size} features each")

dataset_size = len(sensor1_data)
torch.manual_seed(SEED)
indices = torch.randperm(dataset_size, device=DATA_DEVICE)
val_size = int(dataset_size * VAL_SPLIT)
train_size = dataset_size - val_size
val_indices = indices[:val_size]
train_indices = indices[val_size:]
print(f"Split: {train_size} train, {val_size} val")

```

input size 200
Loaded data: 1219813 samples, 200 features each
Split: 975851 train, 243962 val

In [4]:

```

# =====
# MODEL
# =====
class Net(nn.Module):
    def __init__(self, input_size):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_size, 120),
            nn.ReLU(),
            nn.Linear(120, 60),
            nn.ReLU(),
            nn.Linear(60, 30),
            nn.ReLU(),
            nn.Linear(30, 15),
            nn.ReLU(),
            nn.Linear(15, 3)
        )

    def forward(self, x):
        return self.net(x)

net = Net(input_size).to(DEVICE)

```

In [5]:

```

# =====
# LOSS FUNCTION
# =====
def loss_fn(x1, x2, y1, y2, avg_distance):
    # print(type(x1))
    # for i in range(len(x1)):
    #     print(f"{i} {x1[i]} {y1[i]} {x1[i] + y1[i]}")
    # for i in range(len(x2)):
    #     print(f"{i} {x2[i]} {y2[i]} {x2[i] + y2[i]}")
    y1_actual = x1+y1
    y2_actual = x2+y2
    distance = torch.linalg.norm(y1_actual - y2_actual, dim=1)
    constraint = ((distance - avg_distance) ** 2).mean()

    # small regularization to prevent drift

```



```

    reg = 0.01 * ((y1**2).mean() + (y2**2).mean())

    return constraint + reg

# =====
# TRAINING SETUP
# =====
optimizer = optim.Adam(net.parameters(), lr=LEARNING_RATE)

# =====
# TRAINING LOOP
# =====
if mlflow.active_run():
    mlflow.end_run()
mlflow.set_tracking_uri(MLFLOW_TRACKING_URI)
mlflow.set_experiment(MLFLOW_EXPERIMENT)
mlflow.start_run(run_name=MLFLOW_RUN_NAME)
mlflow.log_params({
    "sensor_distance": SENSOR_DISTANCE,
    "batch_size": BATCH_SIZE,
    "epochs": EPOCHS,
    "learning_rate": LEARNING_RATE,
    "val_split": VAL_SPLIT,
    "seed": SEED,
    "input_size": input_size,
})
layer_lines = []
for name, module in net.named_modules():
    if name == "":
        continue
    layer_lines.append(f"{name}: {module.__class__.__name__}")
mlflow.log_text("\n".join(layer_lines), "model_layers.txt")
mlflow.log_param("layer_count", len(layer_lines))

last_avg_loss = 10
best_loss = float("inf")
best_epoch = -1
for epoch in range(EPOCHS):
    perm = torch.randperm(train_size, device=train_indices.device)
    train_perm = train_indices[perm]

    batch_losses = []
    distance_errors = []

    for i in range(0, train_size, BATCH_SIZE):
        indices = train_perm[i:i+BATCH_SIZE]

        batch1 = sensor1_data[indices].to(DEVICE)
        batch2 = sensor2_data[indices].to(DEVICE)
        x1 = sensor1_results[indices].to(DEVICE)
        x2 = sensor2_results[indices].to(DEVICE)
        avg_dist = avg_distance[indices].view(-1).to(DEVICE)

        y1 = net(batch1)
        y2 = net(batch2)

        loss = loss_fn(x1, x2, y1, y2, avg_dist)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        batch_losses.append(loss.item())

    with torch.no_grad():
        y1_actual = x1 + y1
        y2_actual = x2 + y2
        distance = torch.linalg.norm(y1_actual - y2_actual, dim=1)
        distance_errors.extend((distance - avg_dist).cpu().tolist())

    batch_loss_tensor = torch.tensor(batch_losses, dtype=torch.float32)
    avg_loss = batch_loss_tensor.mean().item()
    std_loss = batch_loss_tensor.std(unbiased=False).item()

    distance_error_tensor = torch.tensor(distance_errors, dtype=torch.float32)
    avg_distance_error = distance_error_tensor.abs().mean().item()
    std_distance_error = distance_error_tensor.std(unbiased=False).item()

    with torch.no_grad():
        val_batch_losses = []
        val_distance_errors = []
        for i in range(0, val_size, BATCH_SIZE):
            indices = val_indices[i:i+BATCH_SIZE]

            batch1 = sensor1_data[indices].to(DEVICE)
            batch2 = sensor2_data[indices].to(DEVICE)
            x1 = sensor1_results[indices].to(DEVICE)
            x2 = sensor2_results[indices].to(DEVICE)
            avg_dist = avg_distance[indices].view(-1).to(DEVICE)

```



```

        y1 = net(batch1)
        y2 = net(batch2)

        loss = loss_fn(x1, x2, y1, y2, avg_dist)
        val_batch_losses.append(loss.item())

        y1_actual = x1 + y1
        y2_actual = x2 + y2
        distance = torch.linalg.norm(y1_actual - y2_actual, dim=1)
        val_distance_errors.extend((distance - avg_dist).cpu().tolist())

    val_loss_tensor = torch.tensor(val_batch_losses, dtype=torch.float32)
    val_avg_loss = val_loss_tensor.mean().item() if val_batch_losses else float("nan")
    val_std_loss = val_loss_tensor.std(unbiased=False).item() if val_batch_losses else float("nan")

    val_distance_error_tensor = torch.tensor(val_distance_errors, dtype=torch.float32)
    val_avg_distance_error = val_distance_error_tensor.abs().mean().item() if val_distance_errors else float("nan")
    val_std_distance_error = val_distance_error_tensor.std(unbiased=False).item() if val_distance_errors else float("nan")

    mlflow.log_metrics({
        "train_loss": avg_loss,
        "train_loss_std": std_loss,
        "train_dist_err_mean": avg_distance_error,
        "train_dist_err_std": std_distance_error,
        "val_loss": val_avg_loss,
        "val_loss_std": val_std_loss,
        "val_dist_err_mean": val_avg_distance_error,
        "val_dist_err_std": val_std_distance_error,
    }, step=epoch + 1)

    if val_avg_loss < best_loss:
        best_loss = val_avg_loss
        best_epoch = epoch + 1
        torch.save(net.state_dict(), "best_model.pt")

    if avg_loss > last_avg_loss * 3:
        print(f"Warning: Loss increased significantly from {last_avg_loss:.6f} to {avg_loss:.6f}")
        break
    last_avg_loss = avg_loss
    print(
        f"Epoch {epoch+1}/{EPOCHS}, "
        f"Train Loss: {avg_loss:.6f}, Train Loss std: {std_loss:.6f}, "
        f"Train Dist err mean: {avg_distance_error:.6f}, Train Dist err std: {std_distance_error:.6f}, "
        f"Val Loss: {val_avg_loss:.6f}, Val Loss std: {val_std_loss:.6f}, "
        f"Val Dist err mean: {val_avg_distance_error:.6f}, Val Dist err std: {val_std_distance_error:.6f}"
    )
```

Epoch 1/15, Train Loss: 0.000622, Train Loss std: 0.002888, Train Dist err mean: 0.007105, Train Dist err std: 0.019351, Val Loss: 0.000407, Val Loss std: 0.000244, Val Dist err mean: 0.006819, Val Dist err std: 0.014422
Epoch 2/15, Train Loss: 0.000306, Train Loss std: 0.000328, Train Dist err mean: 0.006151, Train Dist err std: 0.011454, Val Loss: 0.000214, Val Loss std: 0.000113, Val Dist err mean: 0.005693, Val Dist err std: 0.008813
Epoch 3/15, Train Loss: 0.000262, Train Loss std: 0.000182, Train Dist err mean: 0.005806, Train Dist err std: 0.010312, Val Loss: 0.000260, Val Loss std: 0.000136, Val Dist err mean: 0.006059, Val Dist err std: 0.010833
Epoch 4/15, Train Loss: 0.000230, Train Loss std: 0.000161, Train Dist err mean: 0.005571, Train Dist err std: 0.009410, Val Loss: 0.000347, Val Loss std: 0.000190, Val Dist err mean: 0.006270, Val Dist err std: 0.011360
Epoch 5/15, Train Loss: 0.000227, Train Loss std: 0.000165, Train Dist err mean: 0.005484, Train Dist err std: 0.009300, Val Loss: 0.000201, Val Loss std: 0.000108, Val Dist err mean: 0.005349, Val Dist err std: 0.008145
Epoch 6/15, Train Loss: 0.000215, Train Loss std: 0.000145, Train Dist err mean: 0.005381, Train Dist err std: 0.008833, Val Loss: 0.000328, Val Loss std: 0.000222, Val Dist err mean: 0.006036, Val Dist err std: 0.013368
Epoch 7/15, Train Loss: 0.000206, Train Loss std: 0.000136, Train Dist err mean: 0.005309, Train Dist err std: 0.008613, Val Loss: 0.000168, Val Loss std: 0.000095, Val Dist err mean: 0.005059, Val Dist err std: 0.007277
Epoch 8/15, Train Loss: 0.000204, Train Loss std: 0.000150, Train Dist err mean: 0.005238, Train Dist err std: 0.008481, Val Loss: 0.000216, Val Loss std: 0.000146, Val Dist err mean: 0.005424, Val Dist err std: 0.010431
Epoch 9/15, Train Loss: 0.000201, Train Loss std: 0.000148, Train Dist err mean: 0.005179, Train Dist err std: 0.008423, Val Loss: 0.000158, Val Loss std: 0.000098, Val Dist err mean: 0.004850, Val Dist err std: 0.006857
Epoch 10/15, Train Loss: 0.000208, Train Loss std: 0.000161, Train Dist err mean: 0.005243, Train Dist err std: 0.008718, Val Loss: 0.000265, Val Loss std: 0.000142, Val Dist err mean: 0.005556, Val Dist err std: 0.009640
Epoch 11/15, Train Loss: 0.000190, Train Loss std: 0.000127, Train Dist err mean: 0.005113, Train Dist err std: 0.008082, Val Loss: 0.000205, Val Loss std: 0.000119, Val Dist err mean: 0.005222, Val Dist err std: 0.008435
Epoch 12/15, Train Loss: 0.000191, Train Loss std: 0.000131, Train Dist err mean: 0.005070, Train Dist err std: 0.007993, Val Loss: 0.000208, Val Loss std: 0.000113, Val Dist err mean: 0.005205, Val Dist err std: 0.008271
Epoch 13/15, Train Loss: 0.000190, Train Loss std: 0.000135, Train Dist err mean: 0.005046, Train Dist err std: 0.007980, Val Loss: 0.000147, Val Loss std: 0.000085, Val Dist err mean: 0.004814, Val Dist err std: 0.006713
Epoch 14/15, Train Loss: 0.000190, Train Loss std: 0.000158, Train Dist err mean: 0.005029, Train Dist err std: 0.008179, Val Loss: 0.000291, Val Loss std: 0.000175, Val Dist err mean: 0.005596, Val Dist err std: 0.010594
Epoch 15/15, Train Loss: 0.000179, Train Loss std: 0.000118, Train Dist err mean: 0.004998, Train Dist err std: 0.007764, Val Loss: 0.000136, Val Loss std: 0.000081, Val Dist err mean: 0.004667, Val Dist err std: 0.006503

```
In [6]: # =====
# SAVE MODEL + LOG IN TRAINING RUN
# =====
torch.save(net.state_dict(), "model.pt")

os.makedirs(MODEL_SAVE_PATH, exist_ok=True)

# Export only the best model to ONNX.
best_path = "best_model.pt"
best_onnx = f"{MODEL_SAVE_PATH}/best_model.onnx"
if os.path.exists(best_path):
    net.load_state_dict(torch.load(best_path, map_location=DEVICE))
    net.eval()
else:
    print("Warning: best_model.pt not found; exporting current model.")

dummy_input = torch.randn(1, input_size, device=DEVICE)
torch.onnx.export(
    net,
    dummy_input,
    best_onnx,
    export_params=True,
    opset_version=17,
    do_constant_folding=True,
    dynamo=False,
    input_names=["input"],
    output_names=["output"],
    dynamic_axes={"input": {0: "batch_size"}, "output": {0: "batch_size"}},
```



```
)

print(
    f"Exported {best_onnx} from best_model.pt (epoch {best_epoch})."
)

# =====
# LOG + REGISTER MODEL IN MLFLOW

# =====
MODEL_NAME = "BestNN_v1"
if not mlflow.active_run():
    raise RuntimeError("No active MLflow run found. Run training first.")
mlflow.log_param("model_name", MODEL_NAME)
mlflow.log_param("best_epoch", best_epoch)
onnx_model = onnx.load(best_onnx)
mlflow.onnx.log_model(onnx_model, artifact_path="onnx_model")
model_uri = f"runs:/{mlflow.active_run().info.run_id}/onnx_model"
registered = mlflow.register_model(model_uri=model_uri, name=MODEL_NAME)
from mlflow.tracking import MlflowClient
client = MlflowClient()
client.transition_model_version_stage(
    name=MODEL_NAME,
    version=registered.version,
    stage="Production",
)
print(f"Registered model {MODEL_NAME} version {registered.version} (Production)")

mlflow.end_run()
```

/tmp/nix-shell.CPeZ32/nix-shell.MpS9vb/ipykernel_137530/385441134.py:18: DeprecationWarning: You are using the legacy TorchScript-based ONNX export. Starting in PyTorch 2.9, the new torch.export-based ONNX exporter has become the default. Learn more about the new export logic: https://docs.pytorch.org/docs/stable/onnx_export.html. For exporting control flow: https://pytorch.org/tutorials/beginner/onnx/export_control_flow_model_to_onnx_tutorial.html

```
    torch.onnx.export(
2026/04/20 13:52:36 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.
Exported ../models/best_model.onnx from best_model.pt (epoch 15).
```

2026/04/20 13:52:38 WARNING mlflow.utils.environment: Failed to resolve installed pip version. ``pip`` will be added to conda.yaml environment spec without a version specifier.

Registered model 'BestNN_v1' already exists. Creating a new version of this model...

2026/04/20 13:52:38 WARNING mlflow.tracking._model_registry.fluent: Run with id 538898a644d2438a8a7b1588ce1b9d23 has no artifacts at artifact path 'onnx_model', registering model based on models:/m-1cbb3e9f3e7429880e16a369c60b843 instead

2026/04/20 13:52:38 INFO mlflow.store.model_registry.abstract_store: Waiting up to 300 seconds for model version to finish creation. Model name: BestNN_v1, version 2

Created version '2' of model 'BestNN_v1'.

/tmp/nix-shell.CPeZ32/nix-shell.MpS9vb/ipykernel_137530/385441134.py:50: FutureWarning: ``mlflow.tracking.client.MlflowClient.transition\_model\_version\_stage`` is deprecated since 2.9.0. Model registry stages will be removed in a future major release. To learn more about the deprecation of model registry stages, see our migration guide here: <https://mlflow.org/docs/latest/model-registry.html#migrating-from-stages>

```
    client.transition_model_version_stage(
Registered model BestNN_v1 version 2 (Production)
```

 View run traveling-slug-234 at: <http://localhost:5000/#/experiments/1/runs/538898a644d2438a8a7b1588ce1b9d23>

 View experiment at: <http://localhost:5000/#/experiments/1>